

GitHub no Desenvolvimento de Produtos

Conventional Commits

O commit semântico possui os elementos estruturais abaixo (tipos), que informam a intenção do seu commit ao utilizador(a) de seu código.

- **feat** - Commits do tipo feat indicam que seu trecho de código está incluindo um **novo recurso** (se relaciona com o MINOR do versionamento semântico).
- **fix** - Commits do tipo fix indicam que seu trecho de código commitado está **solucionando um problema** (bug fix), (se relaciona com o PATCH do versionamento semântico).
- **docs** - Commits do tipo docs indicam que houveram **mudanças na documentação**, como por exemplo no Readme do seu repositório. (Não inclui alterações em código).
- **test** - Commits do tipo test são utilizados quando são realizadas **alterações em testes**, seja criando, alterando ou excluindo testes unitários. (Não inclui alterações em código)
- **build** - Commits do tipo build são utilizados quando são realizadas modificações em **arquivos de build e dependências**.
- **perf** - Commits do tipo perf servem para identificar quaisquer alterações de código que estejam relacionadas a **performance**.
- **style** - Commits do tipo style indicam que houveram alterações referentes a **formatações de código**, semicolons, trailing spaces, lint... (Não inclui alterações em código).
- **refactor** - Commits do tipo refactor referem-se a mudanças devido a **refatorações que não alterem sua funcionalidade**, como por exemplo, uma alteração no formato como é processada determinada parte da tela, mas que manteve a mesma funcionalidade, ou melhorias de performance devido a um code review.
- **chore** - Commits do tipo chore indicam **atualizações de tarefas** de build, configurações de administrador, pacotes... como por exemplo adicionar um

pacote no gitignore. (Não inclui alterações em código)

- **ci** - Commits do tipo ci indicam mudanças relacionadas a **integração contínua** (*continuous integration*).
- **raw** - Commits do tipo raw indicam mudanças relacionadas a arquivos de configurações, dados, features, parametros.
- **cleanup** - Commits do tipo cleanup são utilizados para remover código comentado, trechos desnecessários ou qualquer outra forma de limpeza do código-fonte, visando aprimorar sua legibilidade e manutenibilidade.
- **remove** - Commits do tipo remove indicam a exclusão de arquivos, diretórios ou funcionalidades obsoletas ou não utilizadas, reduzindo o tamanho e a complexidade do projeto e mantendo-o mais organizado.

Padrões de emojis 🚩

Tipo do commit	Emoji	Palavra-chave
Acessibilidade	 <code>:wheelchair:</code>	
Adicionando um teste	 <code>:white_check_mark:</code>	<code>test</code>
Atualizando a versão de um submódulo	 <code>:arrow_up:</code>	
Retrocedendo a versão de um submódulo	 <code>:arrow_down:</code>	
Adicionando uma dependência	 <code>:heavy_plus_sign:</code>	<code>build</code>
Alterações de revisão de código	 <code>:ok_hand:</code>	<code>style</code>
Animações e transições	 <code>:dizzy:</code>	
Bugfix	 <code>:bug:</code>	<code>fix</code>
Comentários	 <code>:bulb:</code>	<code>docs</code>
Commit inicial	 <code>:tada:</code>	<code>init</code>
Configuração	 <code>:wrench:</code>	<code>chore</code>
Deploy	 <code>:rocket:</code>	
Documentação	 <code>:books:</code>	<code>docs</code>
Em progresso	 <code>:construction:</code>	
Estilização de interface	 <code>:lipstick:</code>	<code>feat</code>
Infraestrutura	 <code>:bricks:</code>	<code>ci</code>

Lista de ideias (tasks)	 :soon:	
Mover/Renomear	 :truck:	chore
Novo recurso	 :sparkles:	feat
Package.json em JS	 :package:	build
Performance	 :zap:	perf
Refatoração	 :recycle:	refactor
Limpeza de Código	 :broom:	cleanup
Removendo um arquivo	 :wastebasket:	remove
Removendo uma dependência	 :heavy_minus_sign:	build
Responsividade	 :iphone:	
Revertendo mudanças	 :boom:	fix
Segurança	 :lock:	
SEO	 :mag:	
Tag de versão	 :bookmark:	
Teste de aprovação	 :heavy_check_mark:	test
Testes	 :test_tube:	test
Texto	 :pencil:	
Tipagem	 :label:	
Tratamento de erros	 :goal_net:	
Dados	 :card_file_box:	raw

Exemplos

Comando Git	Resultado no GitHub
<code>git commit -m ":tada: Commit inicial"</code>	 Commit inicial
<code>git commit -m ":books: docs: Atualização do README"</code>	 docs: Atualização do README
<code>git commit -m ":bug: fix: Loop infinito na linha 50"</code>	 fix: Loop infinito na linha 50
<code>git commit -m ":sparkles: feat: Página de login"</code>	 feat: Página de login
<code>git commit -m ":bricks: ci: Modificação no Dockerfile"</code>	 ci: Modificação no Dockerfile

<code>git commit -m ":recycle: refactor: Passando para arrow functions"</code>	 refactor: Passando para arrow functions
<code>git commit -m ":zap: perf: Melhoria no tempo de resposta"</code>	 perf: Melhoria no tempo de resposta
<code>git commit -m ":boom: fix: Revertendo mudanças ineficientes"</code>	 fix: Revertendo mudanças ineficientes
<code>git commit -m ":lipstick: feat: Estilização CSS do formulário"</code>	 feat: Estilização CSS do formulário
<code>git commit -m ":test_tube: test: Criando novo teste"</code>	 test: Criando novo teste
<code>git commit -m ":bulb: docs: Comentários sobre a função LoremIpsum()"</code>	 docs: Comentários sobre a função LoremIpsum()
<code>git commit -m ":card_file_box: raw: RAW Data do ano aaaa"</code>	 raw: RAW Data do ano aaaa
<code>git commit -m ":broom: cleanup: Eliminando blocos de código comentados e variáveis não utilizadas na função de validação de formulário"</code>	 cleanup: Eliminando blocos de código comentados e variáveis não utilizadas na função de validação de formulário
<code>git commit -m ":wastebasket: remove: Removendo arquivos não utilizados do projeto para manter a organização e atualização contínua"</code>	 remove: Removendo arquivos não utilizados do projeto para manter a organização e atualização contínua

Créditos: Amanda de Oliveira Costa - DSM5